

Trabajo Práctico

TD3: Algoritmos y Estructuras de Datos

Introducción

Un restaurante de alta cocina necesita un sistema eficiente para gestionar sus reservas. Se cuenta con un prototipo funcional que permite registrar reservas, pero sufre de severos problemas de performance que lo hacen inaceptable para uso en producción.

En vistas a una próxima apertura del local, se han determinado los requisitos de complejidad que debe cumplir la aplicación para proveer una experiencia de usuario de primer nivel, y se ha confiado en los talentosísimos estudiantes de TD3 la desafiante tarea de adaptar la implementación actual para cumplir dichos requisitos.

Descripción del sistema

A modo de introducción, se brinda a continuación un breve resumen del funcionamiento deseado y el diseño actual del sistema. **Esto complementa los comentarios en archivos .h del código base, que contienen la documentación del comportamiento de todas las clases y funciones junto con sus complejidades asociadas.**

Funcionalidad

En este sistema, se trabaja con un restaurante (instancia de la clase `Restaurante`) que gestiona las reservas a través de una agenda. La funcionalidad del sistema consiste en:

- registrar nuevas reservas
- consultar la cantidad de reservas activas
- listar las últimas reservas realizadas
- consultar la cantidad de reservas para un día específico
- identificar a los clientes más frecuentes

El proceso de creación de una reserva se realiza a través del método `restaurante.nueva_reserva(...)`, que se encarga de:

1. Crear la reserva con un ID único autoasignado
2. Notificar a la agenda sobre la nueva reserva
3. Retornar el ID asignado para su posterior uso

La clase `Restaurante` provee una interfaz básica para la creación de reservas. La clase `Agenda` es la encargada de mantener las estructuras de datos necesarias para responder eficientemente las siguientes consultas:

- obtener la cantidad total de reservas activas
- obtener las últimas reservas registradas
- obtener la cantidad de reservas en un día específico

- obtener un listado de los clientes más frecuentes

Organización del código

Se recomienda comenzar por inspeccionar los archivos `.h` de las clases **Restaurante** y **Agenda**. Cada uno de estos tiene un archivo `.cpp` asociado con la implementación de su interfaz.

Adicionalmente, se provee el archivo `lib.h` donde se encuentran algunas definiciones utilizadas en todo el programa:

- `struct Reserva`, con la información que compone una reserva
- definición de varios alias de tipos (usando `typedef`) para mejorar la legibilidad

Por último, dentro del directorio `tests` se encuentran las suites de tests asociados a las clases **Restaurante** y **Agenda**. Se recomienda fuertemente leer los tests para ver ejemplos del uso de estas interfaces y su comportamiento esperado.

Trabajando con timestamps

Con *timestamp* nos referimos a la representación de un instante de tiempo, utilizado para marcar el momento en que ocurrió un evento (en este caso, el momento de la reserva). En este sistema, se representan como números utilizando el sistema de **POSIX time**¹.

Para trabajar con timestamps, se proveen funciones auxiliares en `lib.h`:

- `timestamp principio_del_dia(timestamp t)`: devuelve el timestamp correspondiente a la medianoche anterior a t .
- `timestamp fin_del_dia(timestamp t)`: devuelve el timestamp correspondiente a la medianoche siguiente a t .

Tareas a cumplir

Se deben cumplir las siguientes tareas:

1. Modificar la estructura de representación y algoritmos de la clase **Agenda** para cumplir los requisitos de complejidad descritos a continuación.
2. Justificar que el código provisto cumple con las complejidades solicitadas en el **peor caso**.
3. Escribir el invariante de representación de la clase **Agenda**

Requerimientos de complejidad temporal

Se pide respetar las siguientes cotas de complejidad en el peor caso:

<code>registrar_reserva</code>	$O(\log(D) + C)$
<code>cantidad_reservas</code>	$O(1)$
<code>ultimas_reservas</code>	$O(k)$, donde k es la cantidad de reservas a listar
<code>reservas_del_dia</code>	$O(\log(D))$
<code>clientes_frecuentes</code>	$O(k)$, donde k es la cantidad de clientes a listar

¹La convención es que estos números se interpretan como la cantidad de segundos que ocurrieron desde las 00:00hs del 1 de enero de 1970 UTC.

Donde:

D	máxima cantidad de días distintos con reservas
C	máxima cantidad de clientes distintos que realizaron reservas

Cómo justificar la complejidad temporal

Las justificaciones de complejidad deben seguir los siguientes lineamientos:

- La complejidad total de cada método se deben utilizar argumentos basados en álgebra de órdenes
- Se debe anotar cada sección relevante del código con su complejidad correspondiente.
- No es necesario anotar la cantidad exacta de operaciones de cada línea. Por ejemplo, se puede decir directamente que la línea `a = a + 1` tiene $O(1)$ operaciones.
- Está permitido utilizar argumentos basados en complejidad amortizada para justificar la complejidad de ciclos.

Escribiendo el invariante de representación

Se pide escribir el invariante de representación de la clase *Agenda* **tanto en castellano como en lógica de primer orden** (predicado $Rep(e : Agenda)$).

A tener en cuenta:

- La versión en lenguaje natural debe entregarse en un comentario claramente identificado en el archivo `agenda.h`.
- Se deben capturar las condiciones que deben cumplir todos los campos de la clase *Agenda*.
- Se puede asumir como definida la igualdad para el tipo *Reserva*, que será verdadera si todos los campos de dos reservas son iguales.

También cuentan con los siguientes predicados y funciones que no es necesario definir:

- $esPrincipioDeDia(t : \text{int})$
 - será verdadero si y sólo si t es el timestamp asociado al principio de algún día.
- $principio(t : \text{int}) : \text{int}$
 - denota el timestamp asociado al principio del día de t (es decir, la medianoche previa a t para valores que no son ya medianoche).

Trabajando sobre el código

- El código provisto contiene la configuración necesaria para cargar el proyecto en VSCode dentro de un container.
- Todas las modificaciones al código deben limitarse a los archivos `agenda.h` y `agenda.cpp`, modificando sólo la parte privada de la clase (NO su interfaz).
- Se debe respetar el comportamiento documentado.
- Este proyecto no tiene un ejecutable interactivo. Deben enfocarse en que los tests sigan pasando a medida que cambian la implementación.
- Se evaluarán tanto la corrección técnica de la solución como la claridad del código escrito.
- Se recomienda escribir tests adicionales si desean probar otros casos bordes no cubiertos o hacer pruebas y utilizar el debugger para inspeccionar el comportamiento del código si es necesario.

Condiciones de entrega

- La fecha límite de entrega es el **viernes 19 de junio a fin del día**.
- El TP debe realizarse en grupos de 3 personas. Los grupos se deben registrar en el formulario provisto por el cuerpo docente junto con este enunciado.
- Se deben entregar vía campus únicamente tres archivos:
 - `agenda.h` y `agenda.cpp` (siguiendo las indicaciones previas)
 - Un único archivo de imagen o PDF donde se lea **con claridad** la especificación formal del Rep.